PRD: TalentMatch

1. Header

Field	Value
Product	TalentMatch
Project ID / Track	PST_PD_005
Group	G408
Batch / Level	2026 Level 3 (Stretch)
Duration	4 weeks
Student / Team	Fenil Bhavinkumar Patel (PST-26-0025), Aayush Hetalkumar Suthar (PST-26-0014)
Industry	Human Resources / Recruitment Technology
PRD Version	v1.0
Project Type	One-month stretch OJT
Primary Stack	HTML, CSS, JavaScript, Bootstrap 5
Optional Stretch Stack	React + Vite

2. Overview

TalentMatch is a lightweight tech job search aggregator and candidate matching demo. It helps job applicants explore job opportunities and understand how well their skills align with a

job description. It also gives corporate recruiters a simple way to review sample candidate profiles and compare their suitability for a role.

The project is designed as a four-week OJT demonstration, not a production recruitment platform. The team must first build a working application using HTML, CSS, JavaScript, Bootstrap 5, and localStorage or sample JSON data. Optional technologies such as React + Vite or Chart.js should be explored only after the core demo works.

Product promise: An applicant can search sample tech jobs, view a match score with an understandable breakdown, and shortlist a suitable opportunity in under 30 seconds.

Attribute	Description
Product category	Job discovery and candidate matching
Primary users	Job Applicants and Corporate Recruiters
Core problem	Job seekers struggle to compare opportunities against their skills, while recruiters need a quick way to review candidate suitability.
Main value	Bring job discovery, skill matching, and basic candidate review into one simple interface.
Data source	Curated sample jobs and candidate profiles stored locally or loaded from JSON files
Persistence	Browser localStorage or JSON-based sample data
Deployment	GitHub Pages, Vercel, or another suitable static hosting platform
Success condition	The critical user flows work with sample data on laptop and phone-width screens.

Important scope boundary: The MVP does not require integration with real job portals, a production backend, machine learning, or a live candidate-scoring API. Candidate matching must be demonstrated with a transparent JavaScript scoring function or a small optional Python implementation.

3. Problem Statement

Job applicants often search across multiple job listings and manually compare each job's required skills, experience level, location, and role expectations with their own profile. This process can be repetitive and makes it difficult to prioritize relevant opportunities.

Corporate recruiters face a related problem: reviewing multiple candidate profiles against a role can take time, especially when the information is inconsistent or difficult to compare.

TalentMatch addresses these problems through a focused demonstration that combines sample technology job listings, candidate skill profiles, and a simple explainable matching score.

Problems to solve

1. **Scattered job information:** Applicants need a consistent view of job titles, companies, required skills, experience, and locations.
2. **Manual skill comparison:** Applicants have no quick way to see which required skills match their profile and which are missing.
3. **Slow candidate review:** Recruiters need a compact view of candidate skills and suitability for a selected role.
4. **Limited visibility:** Users need clear feedback when no matching jobs or candidates are available.

MVP solution

TalentMatch will provide:

- A searchable catalogue of sample technology jobs.
- A candidate profile containing skills and experience.
- A transparent skill-based match score for each job or candidate.
- A recruiter view for reviewing sample candidates against a selected job.
- Shortlisting and basic status management.
- Local persistence for user-created changes.

Product constraints

- The MVP uses sample data and does not scrape external job websites.
- The score is a heuristic, not a prediction of hiring success.
- The application must not present fabricated matches as verified recommendations.
- The team must not claim to have a production API if scoring runs entirely in the browser.

4. Personas

Persona 1: Job Applicant

Attribute	Details
Example persona	Final-year student applying for entry-level technology roles
Primary goal	Find relevant technology jobs and understand how well their skills match each opportunity.
Main frustration	Repeatedly comparing job descriptions with a resume or skill list.
Key actions	Browse jobs, search by title or skill, view match breakdowns, update skills, shortlist jobs.
Success indicator	Can identify and shortlist a relevant job within 30 seconds using the sample dataset.

Persona 2: Corporate Recruiter

Attribute	Details
Example persona	Recruiter screening applicants for a junior frontend developer position
Primary goal	Review candidate profiles and identify which candidates meet the stated role requirements.
Main frustration	Comparing candidates across separate records without a consistent summary.
Key actions	Select a role, review candidates, inspect matching skills, sort by score, update review status.

Success indicator	Can find and inspect a relevant sample candidate without manually opening every profile.

Persona 3: Project Demonstrator

Attribute	Details
Example persona	Mentor or evaluator assessing the OJT project
Primary goal	Verify that the critical paths, data persistence, responsive layout, and matching logic work.
Main frustration	Demos that depend on unavailable APIs or fail when refreshed.
Key actions	Load sample data, switch views, create or update records, refresh, and test on a phone-width layout.
Success indicator	Can complete the scripted demonstration without setup problems or broken screens.

5. Key User Journeys

Journey A: Applicant finds a relevant job

1. The applicant opens the TalentMatch dashboard.
2. The application displays sample technology jobs.
3. The applicant searches for a role such as “Frontend Developer” or filters by a required skill.
4. The applicant opens a job card or detail panel.
5. TalentMatch compares the job's required skills with the applicant's saved skills.
6. The applicant sees a match score, matched skills, and missing skills.
7. The applicant shortlists the job.
8. The shortlist remains available after refreshing the page.

Expected outcome: The applicant understands why the opportunity is relevant and can save it for later.

Journey B: Recruiter reviews candidates

1. The recruiter opens the recruiter view.
2. The recruiter selects a sample job or role.
3. The application displays the sample candidates.
4. Candidates are ranked by their calculated match score.
5. The recruiter opens a candidate profile.
6. The application shows matched skills, missing skills, and relevant experience information.
7. The recruiter changes the candidate's review status to “Shortlisted,” “Reviewing,” or “Rejected.”
8. The status is saved locally.

Expected outcome: The recruiter can compare candidates using consistent, visible criteria.

Journey C: Applicant updates their skills

1. The applicant opens the profile section.
2. The applicant adds or removes a skill.
3. The application validates the profile input.
4. The applicant saves the updated profile.
5. TalentMatch recalculates the scores for the available jobs.
6. The applicant sees the updated match results.

Expected outcome: Match scores reflect the applicant's current profile rather than stale data.

Journey D: Evaluator verifies persistence

1. The evaluator creates or updates a record.
2. The evaluator refreshes the page.
3. The application restores saved data from localStorage.
4. The evaluator verifies that the record remains available.
5. The evaluator clears the demo data or resets to the original sample dataset when necessary.

Expected outcome: Core interactions remain functional after a page refresh.

6. Scope

6.1 Must Have — MVP and Graded Requirements

Feature	Description	Acceptance Criteria
Tech job search aggregator	Display a catalogue of sample technology jobs in a consistent format.	At least 10 sample jobs are available, with title, company, skills, experience level, location, and description.
Job search	Search jobs by title, company, or skill.	Results update correctly when the user enters a query.
Job details	Display the requirements and details for a selected job.	The user can inspect all relevant fields without navigating to a broken external link.
Applicant profile	Store a basic applicant profile with skills and experience level.	The profile can be viewed and updated.
Candidate match scoring	Calculate a score based on job requirements and candidate skills.	The score is between 0 and 100 and is accompanied by a clear explanation.
Recruiter candidate review	Display sample candidates for a selected role.	At least 5 sample candidates can be compared against a job.
Candidate status	Allow recruiters to update a candidate's review status.	The status can be changed and remains saved after refresh.
Job shortlisting	Allow applicants to shortlist and remove jobs.	Shortlist actions work and remain saved after refresh.
Persistence	Use localStorage or JSON-based sample data.	User-created changes survive refresh when localStorage is used.
Responsive layout	Use Bootstrap 5 and responsive CSS.	Core screens remain usable at phone and laptop widths.

Validation and feedback	Validate forms and display success, empty, and error messages.	Invalid input is handled without breaking the application.
Sample data	Provide realistic, clearly labelled demonstration records.	The app works without requiring an external service or API key.
Documentation	Explain setup, features, data format, and limitations.	A reviewer can run the project using the README.

6.2 Good to Have

Feature	Description	Priority
Advanced filters	Filter by experience level, location, or required skill.	Medium
Sort by match score	Sort jobs or candidates from highest to lowest score.	Medium
CSV export	Export a main job or candidate table to CSV.	Low
PDF export	Export a concise report if implementation is straightforward.	Low
Dashboard summary	Display counts for available jobs, shortlisted jobs, and candidate statuses.	Low
Match breakdown visual	Use simple progress bars to represent matched skills.	Low

6.3 Stretch Goals — Optional Self-Study

Only begin these after the complete MVP works.

Stretch	Description	Condition

React + Vite	Rebuild one existing screen using reusable React components.	Core flows must already work.
Chart.js	Display simple summary charts for jobs or candidate review statuses.	Only if it adds value to the demo.
Python scoring script	Implement the same scoring rule in a small Python program for comparison.	Optional; not a replacement for the required frontend demo.
Simulated scoring API	Create a clearly labelled mock endpoint or a small local Python API that returns match results.	Optional; document setup and dependencies.

6.4 Out of Scope

The following features are excluded from the four-week MVP:

1. **Live external job aggregation:** No scraping, crawling, or integration with commercial job portals is required.
2. **Production recruitment infrastructure:** No real authentication, payment system, production database, or enterprise role-management system.
3. **AI-based hiring predictions:** No machine learning model, automated resume ranking claim, or prediction of hiring success is required.

Real applicant information should not be collected for the demonstration. Use fictional sample profiles.

7. User Stories

ID	User Story	Priority	Acceptance Criteria
TM-01	As an applicant, I want to browse technology jobs so that I can discover opportunities.	Must	Sample job cards display the required job information.
TM-02	As an applicant, I want to search by title, company, or skill so that I can find relevant jobs quickly.	Must	Search returns correct matches and supports a no-results state.

TM-03	As an applicant, I want to view job requirements so that I can decide whether to apply.	Must	Job details show description, required skills, location, and experience level.
TM-04	As an applicant, I want to save my skills so that matching uses my current profile.	Must	Saved skills persist and affect calculated scores.
TM-05	As an applicant, I want to see a match score so that I can compare opportunities.	Must	Score is between 0 and 100 and includes a breakdown.
TM-06	As an applicant, I want to shortlist a job so that I can revisit it.	Must	Shortlist changes persist after refresh.
TM-07	As a recruiter, I want to select a role so that I can review candidates for that opening.	Must	Candidate results correspond to the selected role.
TM-08	As a recruiter, I want to compare candidates so that I can identify relevant profiles.	Must	Candidate rows show a score and key matching information.
TM-09	As a recruiter, I want to inspect matched and missing skills so that I can understand each score.	Must	The candidate detail view lists matched and missing required skills.
TM-10	As a recruiter, I want to update a candidate's review status so that I can track screening progress.	Must	Supported status changes persist after refresh.
TM-11	As a user, I want helpful feedback when no results exist so that I know what to do next.	Must	Empty states provide a useful message and recovery action.
TM-12	As a user, I want the application to work on a phone so that I can use it on smaller screens.	Must	Core actions work at a phone-width viewport.

TM-13	As an evaluator, I want sample data and setup instructions so that I can reproduce the demo.	Must	The README documents setup, seed data, and demo steps.
TM-14	As a recruiter, I want to export the candidate list so that I can review it outside the application.	Good to have	CSV export downloads the displayed records with appropriate columns.
TM-15	As a user, I want to sort results by match score so that I can prioritize stronger matches.	Good to have	Sorting produces the correct order and handles equal scores consistently.

8. Screen Inventory

Screen ID	Screen	Key Elements	Primary Actions
S-01	Dashboard / Job Search	Navigation, search bar, summary cards, job cards, filters	Search, filter, open job
S-02	Job Details	Job title, company, location, experience, required skills, match score	Shortlist, return to results
S-03	Applicant Profile	Name or display label, skills input, experience level, save button	Edit and save profile
S-04	My Shortlist	Saved jobs, score, job title, company, remove action	Open job, remove shortlist
S-05	Recruiter Dashboard	Role selector, candidate count, status summary	Select role, open candidate list
S-06	Candidate Review	Candidate table/cards, scores, search, filters, status selector	Search, sort, change status
S-07	Candidate Details	Candidate skills, experience, matched skills, missing skills, score explanation	Update status, return to list

S-08	Data / Demo Controls	Reset sample data, confirmation message, optional export controls	Reset demo data, export data

Screen and wireframe requirements

Before implementation, the team should create low-fidelity wireframes for all eight screens. A screen may be combined with another when the design remains clear and usable, but all listed user journeys must still be supported.

Wireframes should identify:

- Navigation and active section.
- Primary and secondary actions.
- Search, filtering, and form controls.
- Loading, empty, success, and error states.
- Mobile layout changes.
- Confirmation dialogs for destructive actions.

Wireframe links: Add the actual Figma, Penpot, or other design-tool links to the project README after creating them.

9. Component Breakdown

The application should use reusable components or functions, even if the team uses vanilla JavaScript rather than React.

Component	Responsibility	Inputs / Data	Events / Behaviour
Navbar	Main navigation and current section	Active page, optional user display label	Navigate between applicant and recruiter views
JobCard	Present a single job	Job object, match score, shortlist status	Open details, toggle shortlist
JobSearchBar	Collect a search query	Query string	Update job results

JobFilters	Filter the job list	Location, experience, skills	Apply or clear filters
JobDetailsPanel	Show complete job information	Selected job, match result	Shortlist, close, return
MatchScoreBadge	Show a numeric match score	Score	Display score consistently
SkillBreakdown	Explain matching results	Required, matched, and missing skills	Render skill groups
ApplicantProfileForm	Edit applicant profile	Profile object	Validate, save
CandidateTable	Compare recruiter candidates	Candidate list, scores, statuses	Open details, sort, update status
CandidateCard	Present candidate information on mobile	Candidate object, score, status	Open candidate details
StatusBadge	Represent review status	Status value	Display a consistent status label
SummaryCard	Show a dashboard metric	Label and numeric value	Display summary
EmptyState	Explain that no records match	Message and optional action	Clear filters or return to a useful view
ConfirmationModal	Confirm destructive actions	Title and description	Confirm or cancel
Toast / Alert	Show action feedback	Message and severity	Dismiss or auto-hide

StorageService	Read and write local data	Storage keys and records	Load, save, reset
MatchScoringService	Calculate match scores	Job requirements and candidate profile	Return score and breakdown

Suggested implementation structure

For a vanilla JavaScript implementation:

```
None
talentmatch/
├── index.html
├── pages/
│   ├── applicant.html
│   ├── job-details.html
│   ├── profile.html
│   ├── shortlist.html
│   ├── recruiter.html
│   └── candidate-details.html
├── css/
│   └── styles.css
├── js/
│   ├── app.js
│   ├── jobs.js
│   ├── candidates.js
│   ├── matching.js
```

```
|   ├── storage.js
|   ├── validation.js
|   └── ui.js
├── data/
|   ├── jobs.json
|   └── candidates.json
├── assets/
├── tests/
|   └── demo-checklist.md
└── README.md
```

The team may use a single-page structure instead of separate HTML pages if navigation and state remain manageable. Do not create unnecessary files just to match this example.

10. State and Data

10.1 Core Data Models

Job

```
None
{
  id: "job-001",
  title: "Frontend Developer Intern",
  company: "Sample Tech",
  location: "Bengaluru",
  experienceLevel: "Internship",
  requiredSkills: ["HTML", "CSS", "JavaScript"],
```

```
  description: "Build and maintain responsive user interfaces.",
  source: "Sample Dataset",
  externalUrl: null
}
```

Applicant Profile

```
None
{
  id: "applicant-001",
  displayName: "Demo Applicant",
  skills: ["HTML", "CSS", "JavaScript"],
  experienceLevel: "Entry Level"
}
```

Candidate

```
None
{
  id: "candidate-001",
  displayName: "Candidate A",
  skills: ["HTML", "CSS", "JavaScript"],
  experienceLevel: "Entry Level",
  reviewStatus: "Reviewing"
}
```

Shortlist Record

```
None
{
  jobIds: ["job-001", "job-003"]
}
```

10.2 Suggested localStorage Keys

Key	Purpose	Persistence Rule
`talentmatch_jobs_v1`	Sample or editable job records	Seed from sample JSON when missing
`talentmatch_applicant_v1`	Applicant profile and skills	Save on explicit profile submission
`talentmatch_shortlist_v1`	Shortlisted job IDs	Update whenever a job is added or removed
`talentmatch_candidates_v1`	Sample candidate profiles and review statuses	Preserve status changes
`talentmatch_settings_v1`	Optional UI preferences	Store only preferences that are needed

Sample JSON files may remain the source of truth for the initial dataset. If the application allows changes to jobs or candidates, clearly define how local changes override the seed data.

10.3 Derived Data

Do not store values that can be calculated reliably from the current data unless there is a clear reason.

Derived values include:

- Job match score.
- Matched and missing skills.
- Number of shortlisted jobs.

- Number of candidates in each review status.
- Filtered and sorted job results.
- Filtered and sorted candidate results.

Recalculate derived values when the underlying profile, job requirements, or candidate data changes.

10.4 UI-Only State

Examples include:

- Current screen or active navigation item.
- Current search query.
- Selected job or candidate.
- Active filters and sort order.
- Whether a modal is open.
- Current validation errors.
- Toast visibility.

UI-only state does not need to be persisted unless the team deliberately implements a useful preference.

10.5 Candidate Match Scoring Logic

The scoring system must be simple, consistent, and explainable. The MVP can calculate the score using required-skill overlap.

For a job with required skills:

$$
\text{Match Score} =
\frac{\text{Matched Required Skills}}
{\text{Total Required Skills}}\times 100
$$

Example:

Required Skill	Candidate Has Skill?	Result
HTML	Yes	Matched

CSS	Yes	Matched
JavaScript	Yes	Matched
React	No	Missing

There are three matched skills out of four required skills:

$$
\frac{3}{4}\times100=75
$$

The displayed match score is **75/100**.

Suggested JavaScript implementation:

```
None
function calculateMatch(candidateSkills, requiredSkills) {
  const normalize = (skill) => skill.trim().toLowerCase();

  const candidateSet = new Set(candidateSkills.map(normalize));
  const required = [...new Set(requiredSkills.map(normalize))];

  const matchedSkills = required.filter((skill) =>
    candidateSet.has(skill)
  );

  const missingSkills = required.filter((skill) =>
    !candidateSet.has(skill)
  );
```

```
  const score =
    required.length === 0
      ? 0
      : Math.round((matchedSkills.length / required.length) * 100);

  return {
    score,
    matchedSkills,
    missingSkills
  };
}
```

Implementation notes:

- If no required skills are provided, return a score of `0` and show that there are no requirements to compare. Do not imply a 100% match.
- Normalize skill names to avoid differences caused only by capitalization or surrounding spaces.
- Deduplicate skills before calculating the score.
- Explain that this score measures skill overlap only. It does not assess personality, work quality, or actual hiring probability.
- If experience-level weighting is added, document the formula and keep it optional.

10.6 Persistence and Error Handling

- Wrap localStorage reads and writes in reusable functions.
- Handle missing, malformed, or outdated stored data.
- Provide a reset-to-sample-data action for demonstrations.
- Ask for confirmation before resetting saved data.
- Do not place API keys, passwords, or private candidate information in localStorage.
- Do not store derived scores permanently when they can be recalculated.

11. UI States

Screen / Feature	Loading State	Empty State	Error State	Success State	Partial State
Job Search	Show a short loading indicator only if needed	“No jobs match your search. Try another keyword or clear your filters.”	“We couldn't load the job list. Try reloading the sample data.”	Show job cards and result count	Show available jobs with a note if optional data is unavailable
Job Details	Show a compact placeholder if details are loaded asynchronously	“Select a job to see its details.”	“This job could not be found.”	Display job information and match breakdown	Show available job fields and mark missing optional information
Applicant Profile	Populate saved profile fields	Offer a prompt to add skills	Display validation or storage error	“Profile saved successfully.”	Preserve valid fields and identify fields needing correction
Shortlist	Show saved shortlist entries	“You haven't shortlisted any jobs yet.”	“Your saved list couldn't be read. Restore the sample data or try again.”	Display saved jobs	Show remaining valid jobs if a stored job ID is no longer present

Recruiter Dashboard	Show placeholders only if data needs loading	“No roles or candidates are available.”	Display a recoverable data-loading message	Display candidate counts and role selector	Show available candidates with a warning about missing records
Candidate Review	Display a short loading indicator if required	“No candidates match these filters.”	“Candidate records could not be loaded.”	Show candidate records with scores and statuses	Mark incomplete profiles and explain that their scores may be limited
Candidate Details	Display a compact placeholder if required	“Select a candidate to review.”	“This candidate could not be found.”	Show profile and skill breakdown	Display known information and label missing fields
Export	Indicate progress only if the export is substantial	Explain when there are no records to export	Display a clear export failure message	Trigger the download and confirm it	Export available records and explain omitted incomplete fields

General state requirements

- Never leave the user staring at a blank page when data is unavailable.
- Validation messages should be near the relevant field.
- Preserve valid input when validation fails.
- Make error messages actionable.
- Empty states should provide a next step.
- Do not show loading animations for operations that are effectively instantaneous.

12. Responsiveness

The application must work at a minimum of the following viewport widths.

Device Class	Reference Width	Layout	Required Behaviour
Phone	320–575 px	Single-column layout	Job cards stack vertically; filters use a collapsible panel; candidate tables become cards or an intentionally scrollable table.
Tablet	576–991 px	One- or two-column layout	Summary cards wrap; forms remain readable; navigation fits without horizontal overflow.
Laptop	992–1199 px	Two-column or dashboard layout	Search and filters can appear beside results; tables display key fields.
Large laptop / desktop	1200 px and above	Wider dashboard layout	Use available space without stretching text and tables excessively.

Responsive implementation rules

- Use Bootstrap's responsive grid and utility classes where appropriate.
- Prefer mobile-first CSS.
- Use flexible widths rather than fixed-width cards.
- Ensure forms and buttons are easy to operate with touch.
- Use a compact navigation menu on smaller screens.
- Prevent long job titles, skills, and company names from breaking the layout.
- Keep primary actions visible without forcing excessive horizontal scrolling.
- If a table must scroll horizontally, provide a clear visual indication that more columns are available.
- Test at 320 px, 375 px, 768 px, and 1366 px widths.

13. Visual Design, Accessibility and UX Copy

13.1 Visual Direction

TalentMatch should feel like a clean, modern recruitment dashboard. Prioritize readable job information and understandable matching results over decorative effects.

Element	Recommendation
Primary colour	Deep blue, such as `#1D4ED8`
Main background	Light neutral, such as `#F8FAFC`
Main text	Dark slate, such as `#0F172A`
Secondary text	Slate, such as `#475569`
Success state	Green with a readable text label
Warning state	Amber with a readable text label
Error state	Red with a readable text label
Cards	Consistent padding, restrained borders, and subtle shadows
Typography	System font stack or a readable sans-serif
Spacing	Consistent spacing using Bootstrap utilities
Icons	Use familiar icons with text labels for important actions

Colours are starting points, not a substitute for contrast testing. Avoid using colour alone to communicate a candidate's status or match quality.

13.2 Accessibility Requirements

- Use semantic HTML landmarks and a logical heading hierarchy.

- Associate each form field with a visible label.
- Provide meaningful alternative text for informative images.
- Ensure keyboard navigation works for menus, dialogs, forms, and controls.
- Provide visible focus indicators.
- Use accessible names for icon-only controls.
- Use text labels alongside match-score colours and status badges.
- Ensure form errors are programmatically associated with their fields.
- Use appropriate table headers and accessible labels.
- Maintain a contrast ratio of at least **4.5:1 for normal text** and **3:1 for large text**, following WCAG guidance.
- Avoid relying on hover-only interactions.

13.3 Suggested UX Copy

Situation	Suggested Copy
Search placeholder	“Search by job title, company, or skill…”
Empty job results	“No matching jobs found. Try a different keyword or clear your filters.”
Empty shortlist	“Your shortlist is empty. Save jobs you'd like to revisit.”
Profile saved	“Your profile has been saved.”
Job shortlisted	“Job added to your shortlist.”
Job removed	“Job removed from your shortlist.”
Candidate status updated	“Candidate status updated.”
Missing skills	“Skills to work on”
Match explanation	“This score reflects the required skills found in the profile.”

Sample-data notice	“Demo data only. Jobs and candidate profiles are fictional.”
Reset confirmation	“Reset demo data? Your saved changes will be replaced with the original sample records.”

Keep messages short, specific, and useful. Avoid misleading claims such as “You're guaranteed to get this job” or “Best candidate confirmed.”

14. Tech Stack

Technology	Purpose	Requirement
HTML5	Page structure and semantic content	Must have
CSS3	Styling and responsive custom layouts	Must have
JavaScript	DOM interactions, forms, search, filtering, matching, and state updates	Must have
Bootstrap 5	Responsive grid, navigation, cards, forms, modals, and utilities	Must have
localStorage	Persist applicant profile, shortlist, and review statuses	Must have if this persistence option is selected
JSON	Store and load initial job and candidate records	Must have as sample-data format or equivalent local data source
Git and GitHub	Version control, collaboration, and source hosting	Must have
Python basics, OOP and files	Optional supporting scoring or data-processing script	Only if required by the mentor or selected as an extension
React + Vite	Optional component-based rebuild	Stretch only

Chart.js	Optional dashboard charts	Stretch only

Core technical focus

- HTML structure and accessible forms.
- CSS Flexbox, Grid, and Bootstrap responsive utilities.
- JavaScript functions, arrays, objects, Sets, DOM manipulation, and events.
- Search, filter, and sort operations.
- Form validation.
- `fetch()` for loading local JSON data where appropriate.
- localStorage for browser persistence.
- Git branches, commits, and GitHub collaboration.
- Problem solving using data structures, especially arrays, objects, Sets, and Maps.

API clarification

The project description includes a "candidate match scoring API," but the required core stack is frontend-focused. The team must implement the scoring functionality as a reusable JavaScript function and integrate it into the UI.

If a true HTTP API is implemented as a stretch, it must expose an actual endpoint and be documented separately. A local JavaScript function must not be described as a deployed API.

Suggested Git workflow

1. Create one shared GitHub repository.
2. Agree on the folder structure and naming conventions.
3. Create small feature branches for separate tasks.
4. Commit working changes with descriptive messages.
5. Open pull requests and review each other's changes.
6. Merge only after testing the affected flow.
7. Keep the main branch deployable.

15. Learning Goals

15.1 Course Alignment

Course / Area	Application in TalentMatch

Frontend Development Basics	Build pages, forms, navigation, responsive layouts, and reusable UI patterns.
Problem Solving with Data Structure	Store and search jobs and candidates, remove duplicate skills, calculate match scores, and sort results.
JavaScript Fundamentals	Implement events, functions, validation, filtering, persistence, and derived state.
Python Basics, OOP and Files	Optional extension for reading sample records or implementing a small scoring script.
Git and GitHub	Collaborate through branches, commits, reviews, and deployment.
Product Thinking	Define a narrow MVP, test user journeys, document limitations, and avoid unnecessary features.

15.2 Expected Technical Skills

By the end of the project, students should be able to:

- Build a responsive multi-view web application using HTML, CSS, JavaScript, and Bootstrap 5.
- Work with arrays of objects representing jobs and candidates.
- Implement search, filtering, sorting, and shortlisting.
- Use a Set or Map where it simplifies skill comparison or record lookup.
- Calculate a matching score and explain its result.
- Persist data using localStorage and handle missing or invalid saved data.
- Validate user input and provide helpful feedback.
- Test critical user journeys systematically.
- Collaborate using Git and GitHub.
- Deploy and demonstrate a working application.
- Clearly distinguish a frontend prototype from a production service.

15.3 Stretch Learning

React + Vite, Chart.js, and a Python scoring API are optional. They are not prerequisites for a successful MVP, and students should not begin them until the core application is demonstrably working.

16. Milestones

The project lasts **four weeks**. Month 2 and Month 3 are not part of this project schedule.

Week	Focus	Tasks	Expected Deliverable	Exit Criteria
Week 1	Plan and establish the UI	Finalize the one-page brief, wireframes, repository, Bootstrap layout, and sample data structure. Build the first working job-search screen.	Initial repository, wireframes, responsive layout, sample JSON, first screen	The application opens locally and displays sample jobs.
Week 2	Complete the core flows	Implement search, job details, applicant profile, skill matching, recruiter candidate review, status updates, and persistence.	Functional MVP	Applicant and recruiter flows work with sample data and survive refresh where applicable.
Week 3	Test and polish	Validate forms, implement empty/error states, test scoring edge cases, refine mobile layouts, write README, and prepare deployment.	Feature-complete demo, README, test checklist	Critical paths pass on laptop and phone-width screens.
Week 4	Deploy and demonstrate	Fix bugs, review code, test reset behaviour, deploy the demo, capture screenshots, and prepare a short demonstration. Attempt a stretch feature only if time remains.	Deployed demo, screenshots, demo video or live demo, final repository	Evaluator can run the scripted demonstration without external credentials or unavailable services.

Week 3 Stretch Rule

React + Vite, Chart.js, or a Python scoring endpoint may be attempted only when:

- The core applicant journey works.
- The core recruiter journey works.
- Matching scores are explainable and tested.
- Data persistence works.
- The application is usable on a phone-width screen.

If any core requirement remains broken, spend Week 3 fixing it rather than starting a stretch feature.

17. Evaluation

17.1 Evaluation Criteria

Evaluation Area	Weight	What Will Be Checked
Core functionality	30%	Job catalogue, search, job details, applicant profile, matching, recruiter candidate review, and status updates.
Matching logic and data handling	20%	Correct scores, matched/missing skills, search/filter correctness, data structures, and edge cases.
UI and responsiveness	15%	Clear information hierarchy, responsive Bootstrap layout, usable controls, and phone-width behaviour.
Persistence and UI states	10%	localStorage or chosen data persistence, refresh behaviour, validation, empty states, and error recovery.
Code quality and collaboration	10%	Readable functions, sensible file structure, meaningful Git history, and evidence of team contribution.
Documentation and deployment	10%	README, setup instructions, screenshots, deployed demo, sample data, and known limitations.

Demo and usability	5%	Clear walkthrough of critical paths and successful completion of the scripted demo.
Total	**100%**	

17.2 Required Acceptance Tests

Test ID	Test	Expected Result
AT-01	Open the application with the original sample dataset.	Job and candidate records display correctly.
AT-02	Search for a known job title.	Relevant jobs are returned.
AT-03	Search for a nonexistent term.	A useful empty state appears.
AT-04	Open a job and calculate its skill match.	Score, matched skills, and missing skills are consistent.
AT-05	Update the applicant's skills.	The profile is saved and relevant scores recalculate.
AT-06	Shortlist a job and refresh the page.	The job remains shortlisted.
AT-07	Select a recruiter role.	The corresponding candidate list appears.
AT-08	Update a candidate's review status and refresh.	The updated status remains saved.
AT-09	Test a job with no required skills.	The application shows a zero score or an explicit unavailable state, with a clear explanation.

AT-10	Load malformed or missing saved data.	The application recovers gracefully or offers a reset option.
AT-11	Use the application at 320–375 px width.	Primary actions remain usable without layout breakage.
AT-12	Reset the demo dataset.	The user is asked to confirm and the sample dataset is restored.

17.3 Role and Data Integrity Checks

The demo may provide applicant and recruiter views without implementing real authentication. However:

- The UI must not imply that a visual role selector provides secure access control.
- If a role switcher is used, label it as a demo-mode switcher.
- The application should prevent accidental or invalid status values through validation.
- If role-specific actions are simulated, document that they are client-side demonstration behaviour and not secure authorization.

17.4 Expected Output

A working TalentMatch demonstration built on the Semester 1 frontend stack, including:

- Applicant job search and shortlisting.
- Explainable skill-based candidate matching.
- Recruiter candidate review and status management.
- Sample jobs and fictional candidate profiles.
- Local persistence or a clearly documented JSON-based alternative.
- Responsive laptop and phone-width interfaces.
- GitHub repository, README, screenshots, and deployed demo.

18. Risks and Assumptions

Risk / Assumption	Likelihood	Impact	Mitigation
Students attempt live scraping of job websites.	Medium	High	Use a curated sample dataset for the MVP; external integrations are out of scope.

The team spends too much time learning React.	Medium	High	Complete the vanilla JavaScript MVP first and defer React until the stretch gate is met.
Scores are misunderstood as hiring predictions.	Medium	High	Display the scoring formula and label the score as a skill-overlap indicator.
localStorage contains malformed or outdated data.	Medium	Medium	Validate stored data and provide a safe reset-to-sample-data option.
No backend is available for a claimed API.	High	Medium	Implement a JavaScript scoring function first; clearly separate any optional real API.
Matching results are inconsistent because of skill formatting.	Medium	Medium	Normalize case and whitespace, and deduplicate skills before scoring.
The interface breaks on smaller devices.	Medium	Medium	Test phone widths early and use Bootstrap responsive utilities.
Recruiter functionality becomes too complex.	Medium	High	Limit it to role selection, candidate comparison, score breakdown, and review status.
Sample data is mistaken for real job listings.	Low	Medium	Label sample records clearly and avoid misleading external application links.
Team members overwrite each other's code.	Medium	Medium	Use feature branches, small commits, and pull-request reviews.
Deployment fails near the deadline.	Medium	Medium	Deploy a basic working version during Week 2 or early Week 3 and verify it regularly.

Real personal information is used in demo records.	Low	High	Use fictional names and sample data only.

Assumptions

1. The team consists of two students.
2. The project is expected to be completed in four weeks.
3. The main objective is to demonstrate frontend and problem-solving skills.
4. External job portals and live recruitment services are not required.
5. The project must work without API keys or paid services.
6. A JavaScript scoring function satisfies the core matching requirement unless the mentor explicitly requires a separate HTTP API.
7. React + Vite and other listed stretch technologies are optional.

19. Deliverables

Deliverable	Required Content	Completion Check
Product brief and PRD	Problem, personas, user journeys, scope, and acceptance criteria	Final PRD committed to the repository
Wireframes	Designs for all core screens, including mobile variants	Design links included in README
Source code	HTML, CSS, JavaScript, Bootstrap 5, and any approved optional additions	Project runs using documented instructions
Sample data	Fictional job listings and candidate profiles	Application runs without external API credentials
Match scoring	Reusable scoring function and documented formula	Scores and breakdowns pass test cases
Persistence	localStorage implementation or documented JSON alternative	Data behaviour is verified

GitHub repository	Source code and meaningful commit history	Repository is accessible to the evaluator
README	Setup, features, data model, scoring formula, limitations, and screenshots	A new reviewer can follow the instructions
Responsive screenshots	At least one laptop and one phone-width screenshot	Screenshots included in README
Test checklist	Critical-path tests, edge cases, and results	Checklist is completed before final demo
Usability notes	Feedback from a small number of test users or peers and improvements made	Notes are documented
Deployed demo	GitHub Pages, Vercel, or another suitable hosting platform	Deployment opens and supports the main flows
Demo video or live walkthrough	Applicant journey, recruiter journey, scoring, and persistence	All required flows are demonstrated

README must include

- Project overview and problem statement.
- Student names and project ID.
- Features implemented.
- Technologies used.
- Local setup instructions.
- How sample data is loaded.
- How the match score is calculated.
- How persistence works.
- Test instructions.
- Deployment URL.
- Screenshots for laptop and phone-width views.
- Known limitations, including the use of fictional data and the absence of live job-portal integration.

20. References

Resource	Purpose	Link
MDN HTML	Semantic HTML, forms, and page structure	https://developer.mozilla.org/en-US/docs/Web/HTML
MDN CSS	Layout, responsive design, Flexbox, and Grid	https://developer.mozilla.org/en-US/docs/Web/CSS
MDN JavaScript	Functions, arrays, objects, Sets, Maps, and events	https://developer.mozilla.org/en-US/docs/Web/JavaScript
MDN Fetch API	Loading local or remote data where appropriate	https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
MDN localStorage	Browser-based data persistence	https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage
Bootstrap 5	Responsive grid, cards, forms, navigation, and modals	https://getbootstrap.com/docs/5.3/
Python Tutorial	Optional Python fundamentals and data handling	https://docs.python.org/3/tutorial/
Git Documentation	Version control fundamentals	https://git-scm.com/doc
GitHub Docs	Repositories, branches, commits, and pull requests	https://docs.github.com/
GitHub Pages	Static website deployment	https://docs.github.com/en/pages
Vercel Documentation	Application deployment	https://vercel.com/docs

React Documentation	Optional React self-study	https://react.dev/
Vite Documentation	Optional React + Vite setup	https://vite.dev/
Chart.js Documentation	Optional dashboard charts	https://www.chartjs.org/docs/latest/
W3C WCAG	Accessibility and contrast guidance	https://www.w3.org/WAI/standards-guidelines/wcag/

21. Sign-off

Role	Name	Date	Approved
Student	Fenil Bhavinkumar Patel (PST-26-0025)	Pending	Pending
Student	Aayush Hetalkumar Suthar (PST-26-0014)	Pending	Pending
Mentor / Evaluator	To be assigned	Pending	Pending

Final project rule: TalentMatch is a four-week stretch OJT. A small, working, well-tested product is better than an unfinished application with many advanced features. The team must prioritize the job-search journey, explainable matching, recruiter review, persistence, responsive design, and a reliable demonstration before attempting any stretch technology.